13. ROS 2 Distributed Communication

1. Concept

Multi-machine communication, that is, distributed communication, refers to a communication strategy that enables data exchange between different hosts through a network.

ROS 2 itself is a distributed communication framework, which can conveniently implement communication between different devices. The middleware on which ROS 2 is based is DDS. When in the same network, distributed communication can be implemented through the domain ID mechanism of DDS (ROS_DOMAIN_ID). The general process is: before starting a node, you can set the value of the domain ID. If different nodes have the same domain ID, they can freely discover each other and communicate; conversely, if the domain ID values are different, it cannot be implemented. By default, the domain ID used when all nodes start is 0. In other words, as long as you ensure they are in the same network, you do not need to do any configuration, and different nodes on different ROS 2 devices can implement distributed communication.

The application scenarios of distributed communication are relatively wide: unmanned vehicle formation, drone formation, remote control, and so on. The data exchange in all these relies on distributed communication.

2. Implementation

2.1. Default Implementation

You only need to put the host and the slave(s) 【there can be multiple】 in the same network, and distributed communication works. For example, the host and the slave are connected to the same Wi-Fi or the same router.

In Windows, setting the virtual machine network to 【bridged mode】 puts it in the same network as the host.

Test:

- Here we assume that we have two hosts A and B. They can be hosts that can connect to the network in any form, for example: a virtual machine, Raspberry Pi, jetson, x86/ARM host, card mainboard. They only need to have the same version of the ros2 environment installed.

1. Execute on host A:

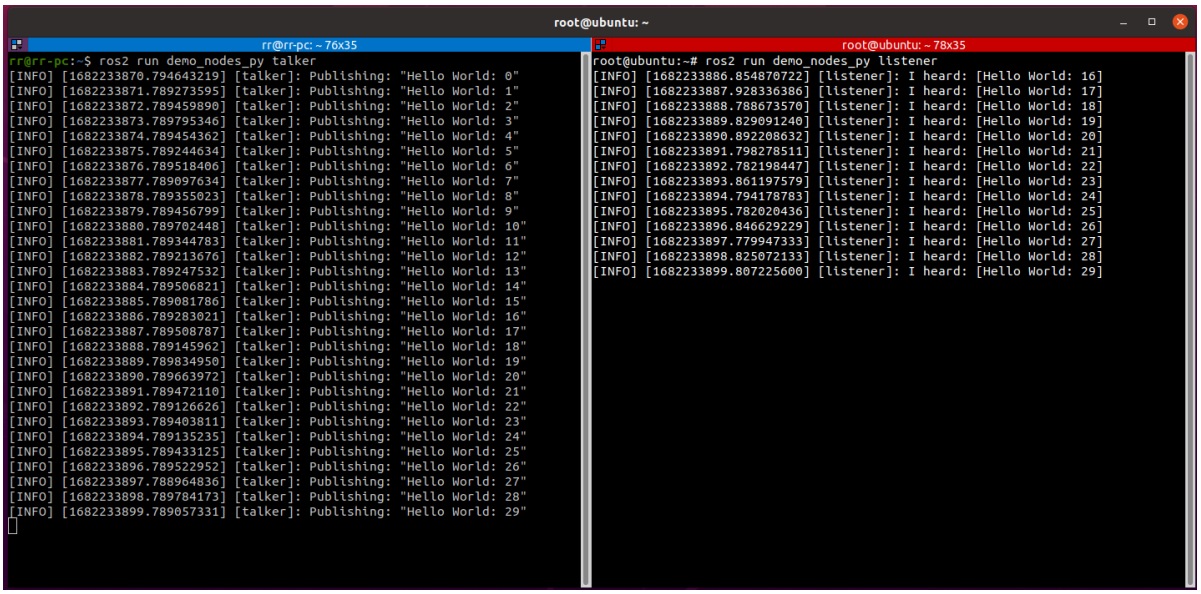
This demo runs the car in Docker with host network mode. In simple terms, host mode means sharing the same network as the car, so there is no difference from executing on the car.

```
ros2 run demo_nodes_py talker
```

2. Execute on host B:

```
ros2 run demo_nodes_py listener
```

If the following is displayed: the topic published by the host can be subscribed to by the slave in time, which means multi-machine communication has been implemented.



2.2. Distributed Network Grouping

Suppose there are other robots in use in the network you are currently in. To avoid interference from other robots, you can also set a group for your robot.

ROS 2 provides a DOMAIN mechanism, which is similar to grouping. Only computers in the same DOMAIN can communicate. We can add the following configuration line to the .bashrc of the host 【the car】 and the slave 【the virtual machine】 to assign the two to the same group:

```
$ export ROS_DOMAIN_ID=<your_domain_id>
```

If the ID assigned to the host 【the car】 and the slave 【the virtual machine】 is different, the two cannot communicate, thus achieving the purpose of grouping.

2.2.1. Case 1

1. Execute on the host 【the car】:

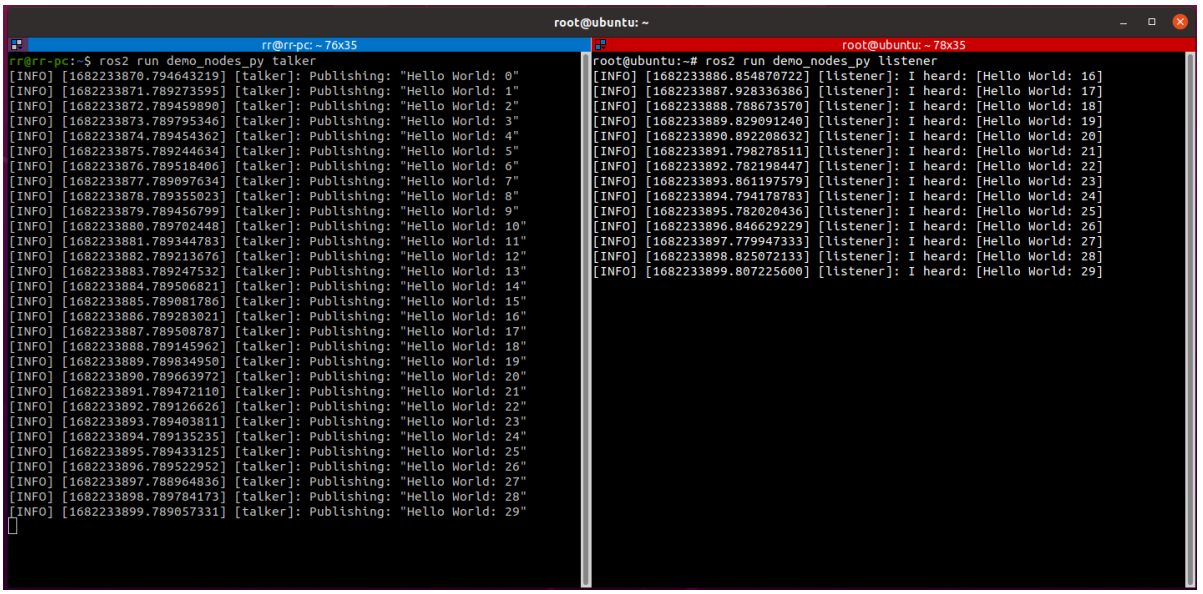
This demo runs the car in Docker with host network mode. In simple terms, host mode means sharing the same network as the car, so there is no difference from executing on the car.

```
echo "export ROS_DOMAIN_ID=6" >> ~/.bashrc  # Here 6 is the ROS_DOMAIN_ID. It
does not have to be 6, as long as it conforms to the rules of ROS_DOMAIN_ID
source ~/.bashrc
ros2 run demo_nodes_py talker
```

2. At the same time, execute on the slave 【the virtual machine】:

```
echo "export ROS_DOMAIN_ID=6" >> ~/.bashrc  # Here the value is kept the same as
on the host
source ~/.bashrc
ros2 run demo_nodes_py listener
```

If the following is displayed: the topic published by the host can be subscribed to by the slave in time, which means grouped multi-machine communication has been implemented



2.2.2. Case 2

Control the movement of the turtlesim through distributed communication

- Host A runs the command

```
ros2 run turtlesim turtlesim_node
```

- Host B runs the command

```
ros2 run turtlesim turtle_teleop_key
```

3. Notes

Setting the value of ROS_DOMAIN_ID is not arbitrary, and there are certain constraints:

1. It is recommended that the value of ROS_DOMAIN_ID be between [0,101], including 0 and 101;
2. The total number of nodes in each domain ID is limited and needs to be less than or equal to 120;
3. If the domain ID is 101, the total number of nodes in that domain needs to be less than or equal to 54.

4. DDS Domain ID Value Calculation Rules (Advanced Knowledge)

The relevant calculation rules for domain ID values are as follows:

1. DDS is based on the TCP/IP or UDP/IP network communication protocol. Network communication requires a specified port number, which is represented by a 2-byte unsigned integer, and its value range is between [0,65535];
2. The allocation of port numbers also has its rules and cannot be used arbitrarily. According to the DDS protocol, 7400 is used as the starting port, so the available ports are [7400,65535]. It is also known that, according to the DDS protocol, each domain ID occupies 250 ports by default. Then the number of domain IDs is: (65535-7400)/250 = 232, and the corresponding value range is [0,231];

3. The operating system also sets some reserved ports. When using ports in DDS, these reserved ports also need to be avoided to prevent conflicts during use. The reserved ports differ among different operating systems. The final result is that in Linux, the available domain IDs are [0,101] and [215-231], and in Windows and Mac, the available domain IDs are [0,166]. In summary, for compatibility with multiple platforms, it is recommended that the domain ID take a value within the range of [0,101].
4. Each domain ID occupies 250 ports by default, and each ROS 2 node needs to occupy two ports. In addition, according to the DDS protocol, within the port segment of each domain ID, the 1st and 2nd ports are the Discovery Multicast port and the User Multicast port, and starting from the 11th and 12th ports are the Discovery Unicast port and the User Unicast port of the first node in the domain. The ports occupied by subsequent nodes follow in sequence. Then the maximum number of nodes in one domain ID is: (250-10)/2 = 120;
5. Special case: when the domain ID value is 101, the latter half of its ports belong to the reserved ports of the operating system, and the maximum number of nodes is 54.

It is sufficient to understand the above calculation rules.